NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

ATTRIBUTE GRAMMARS

IN OCAMLFLAT/OFLAT

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
February, 2025

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

DEPARTMENT OF
COMPUTER SCIENCE

ATTRIBUTE GRAMMARS

IN OCAMLFLAT/OFLAT

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

Adviser: Artur Miguel Dias
Professor Auxiliar, DI-FCT, Universidade NOVA de Lisboa

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
February, 2025

Attribute Grammars
in OCamlFLAT/OFLAT

Copyright © Pedro Emanuel Inácio Bailão, NOVA School of Science and Technology, NOVA University Lisbon.
The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical boundaries, to file and publish this dissertation through printed copies reproduced on paper or on digital form, or by any other means known or that may be invented, and to disseminate through scientific repositories and admit its copying and distribution for non-commercial, educational or research purposes, as long as credit is given to the author and editor.

This document was created with the (pdf/Xe/Lua)LaTeX processor and the NOVAthesis template (v7.1.29) [6].

Abstract

Theoretical concepts in Formal Languages and Automata Theory (FLAT) are known for their formal, rigorous, and often complex nature, making them challenging to learn. To address this challenge, various tools have been developed over the years, allowing users to engage with these concepts through practical exercises and examples.

At FCT-UNL, two pedagogical tools were started to aid in this learning process: the OCamlFLAT library, which provides logical support for FLAT concepts, and the OFLAT graphical application, a web-based tool offering interactive visualizations of these ideas.

The first goal of this dissertation is to extend the OCamlFLAT library by incorporating support for attribute grammars. The second goal is to enhance the OFLAT graphical application by adding features that allow for the visualization and interaction with these grammar concepts.

A key focus of this work is the pedagogical enhancement of the developed software. Consequently, a portion of the dissertation will be devoted to explaining the theoretical foundations of attribute grammars, including various classic variants.

Keywords: Formal Languages, Automata Theory, OCaml, OFLAT, OCamlFLAT, Functional Programming, Formal Grammars

Resumo

Os conceitos teóricos em Teoria de Linguagens Formais e Autómatos (FLAT) são conhecidos pela sua natureza formal, rigorosa e muitas vezes complexa, o que os torna desafiantes para aprender. Para mitigar essa dificuldade, várias ferramentas foram desenvolvidas ao longo dos anos, permitindo que os utilizadores interajam com esses conceitos através de exercícios e exemplos práticos.

Na FCT-UNL, foram criadas duas ferramentas pedagógicas com este propósito: a biblioteca OCamlFLAT, que fornece suporte lógico para os conceitos de FLAT, e a aplicação gráfica OFLAT, uma ferramenta web que oferece visualizações interativas desses conceitos.

O primeiro objetivo desta dissertação é estender a biblioteca OCamlFLAT, incorporando suporte para gramáticas com atributos. O segundo objetivo é melhorar a aplicação gráfica OFLAT, adicionando funcionalidades que permitam a visualização e interatividade com estes conceitos de gramáticas.

Um foco importante deste trabalho é o reforço do caráter pedagógico do software a ser desenvolvido. Assim, uma parte significativa da dissertação será dedicada à explicação dos fundamentos teóricos das gramáticas com atributos, incluindo as suas diferentes variantes clássicas.

Palavras-chave: Teoria de Linguagens Formais, Autómatos, OCaml, OFLAT, OCamlFLAT, Programação Funcional, Gramáticas Formais

Contents

List of Figures **ix**

1 Introduction **1**
1.1 Context 1
1.2 Objectives 1
1.3 Document Structure 1

2 Theory of Computation **3**
2.1 Chomsky Hierarchy 3
2.1.1 Type 0: Unrestricted Grammars 3
2.1.2 Type 1: Monotonic Grammars 4
2.1.3 Type 2: Context-Free Grammars 5
2.1.4 Type 3: Regular Grammars 5
2.2 Attribute Grammars 6
2.2.1 Synthesized Attributes 6
2.2.2 Inherited Attributes 7
2.2.3 Circularity 8
2.3 Constrained Attribute Grammars 9
2.3.1 Example of a Constrained Attribute Grammar 10
2.4 Automata 11
2.4.1 Types of Automata 11

3 Grammar Tools **13**
3.1 JFLAT 13
3.2 ANTLR 14
3.3 CGM: Context-Free Grammar Manipulator 14

4 OCaml-Flat and OFLAT Systems **15**
4.1 OCamlFLAT 15
4.2 OFLAT Platform 16

5 Technology **17**
5.1 Programming with OCaml 17
5.1.1 Declarative Programming 17
5.1.2 Pattern Matching 18
5.2 OCaml/JavaScript Interoperability 18
5.2.1 JavaScript Types 18
5.2.2 Bindings 19
5.2.3 `js_of_ocaml` Modules 19
5.3 Representation in OCaml 20
5.3.1 Synthesized Attributes 20
5.3.2 Inherited Attributes 20
5.3.3 Rules 20
5.4 Cytoscape 21

6 Planning **23**
6.1 Introduction 23
6.2 Support for Attribute Grammars in the OCamlFLAT Library (8 weeks) . 23
6.3 Support for Grammars in the OFLAT Web Application (6 weeks) 24
6.4 Pedagogical Exercises (1 week) 24
6.5 Evaluation (1 week) 24
6.6 Dissertation Writing (8 weeks) 24
6.7 Gantt Chart 25

Bibliography **27**

List of Figures

2.1 Chomsky hierarchy 4
2.2 Attribute grammar for signed binary numbers [3] 9
2.3 Attribute dependency graph for the grammar in figure 2 [3] 9
2.4 Modified attribute dependency graph with circularity 9
2.5 Production rules fi r a CAG for simple mathematical expressions [9] 10

1

Introduction

1.1 Context

The study of Formal Languages and Automata Theory (FLAT) is integral to Computer Engineering and Computer Science programs. However, due to its abstract and mathematical nature, students often struggle to fully grasp the concepts and models involved. Therefore, a tool that complements the study of FLAT theory by being practical, user-friendly, and visually oriented is highly beneficial.

In recent years, the OCamlFLAT/OFLAT tool has been developed at FCT-UNL, offering extensive support in this field. Despite its advanced state, with support for several key models, there are still many features that could enhance its functionality.

This dissertation aims to introduce new capabilities to OCamlFLAT/OFLAT, enhancing both the OCamlFLAT library and the OFLAT graphical interface.

1.2 Objectives

The primary objective of this dissertation is to incorporate support for attribute grammars. Attribute grammars extend context-free grammars by allowing additional information (attributes) to be associated with production rules. These attributes are crucial for describing the semantic properties of the languages generated by the grammars.

To achieve this, the OCamlFLAT library needs to be extended to support attribute grammars. Additionally, graphical enhancements are required to enable the creation and editing of these grammars within the OFLAT platform.

1.3 Document Structure

This thesis is structured into multiple chapters, each covering essential aspects of the research, development, and implementation of the OCamlFLAT library and the OFLAT platform. The document follows a logical progression from theoretical background to practical applications. Below is an overview of its structure:

- **Introduction**: Provides an overview of the dissertation's context, objectives, and structure.
- **Theory of Computation**: Lays the theoretical foundation for the study of FLAT models. It begins with an exploration of the Chomsky hierarchy and progresses into discussions of the four language types (Type 0, 1, 2, and 4).
- **Grammar Tools**: Examines existing pedagogical tools that support the teaching and learning of FLAT theory, with a focus on three prominent systems: JFLAT, ANTLR, and CGM.
- **OCamlFLAT and OFLAT Systems**: Introduces the OCamlFLAT library and the OFLAT platform, discussing their respective roles in the toolset.
- **Technology**: Covers the technical aspects of developing the OCamlFLAT/OFLAT systems.
- **Planning**: In this chapter we define and scheduled the tasks to be accomplished in this project.

2

Theory of Computation

In Theory of Computation, various models of computation with different levels of expressive power are studied. To discuss and compare different levels of expressive power, it is inevitable to consider algorithms in the context of some application domain. The typically chosen application domain is the syntax of formal languages. There are two techniques for defining languages that are typically used. The first technique is based on the concept of generation and uses the formalism of grammars. Another technique is based on the mechanism of recognition and uses the formalism of automata.

2.1 Chomsky Hierarchy

Noam Chomsky (with the help of Marcel-Paul Schützenberger) identified several classes of formal languages, with different levels of specification complexity, requiring computational models with different levels of expressive power to solve the problem of recognizing these languages [1].

There are four levels: type 0, type 1, type 2, and type 3.

- Type 0 — Unrestricted Grammars
- Type 1 — Context-Sensitive Grammars
- Type 2 — Context-Free Grammars
- Type 3 — Regular Grammars

The Chomsky hierarchy involves four levels of language types. Each type of language is associated with a type of generator and a type of recognizer.

2.1.1 Type 0: Unrestricted Grammars

Type 0 grammars, also known as unrestricted grammars, are the most general class in the Chomsky hierarchy. They have no restrictions on their production rules, meaning that any string of symbols can be replaced by any other string of symbols. This flexibility allows

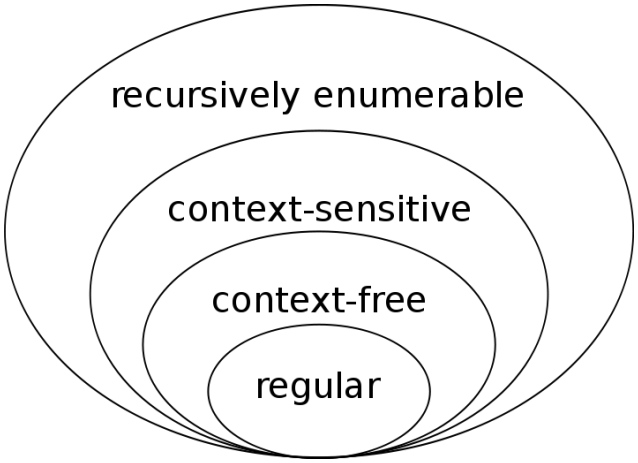


Figure 2.1: Chomsky hierarchy

Type 0 grammars to generate any language that is recognizable by a Turing machine, making them equivalent in power to Turing machines. In other words, they can describe all recursively enumerable languages [1].

An example of an unrestricted grammar is:

$$G = (\{S, A, B\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow aAB, AB \rightarrow BA, A \rightarrow a, B \rightarrow b\}$$

Derivation of the word "aba":

$$\begin{aligned} S &\rightarrow aAB \\ aAB &\rightarrow aBA \\ aBA &\rightarrow abA \\ abA &\rightarrow aba \end{aligned}$$

2.1.2 Type 1: Monotonic Grammars

On type 1 grammars, or monotonic grammars, the length of the string on the left-hand side of a production rule is less than or equal to the length of the string on the right-hand side. This property ensures that the derivation process does not shorten the string. Type 1 grammars can generate languages that are recognized by a linear bounded automaton, which is a Turing machine with a tape of limited length proportional to the input size [1].

An example of a monotonic grammar is:

$$G = (\{S, B\}, \{a, b, c\}, P, S)$$

$$P = \{S \rightarrow aBSc, S \rightarrow abc, Ba \rightarrow aB, Bb \rightarrow bb\}$$

Derivation of the word "aaabbbccc":

$$
\begin{aligned}
S &\longrightarrow aBSc \\
aBSc &\longrightarrow aBaBScc \\
aBaBScc &\longrightarrow aBaBabccc \\
aBaBabccc &\longrightarrow aBaaBbccc \\
aBaaBbccc &\longrightarrow aaBaBbccc \\
aaBaBbccc &\longrightarrow aaaBBbccc \\
aaaBBbccc &\longrightarrow aaabbbccc
\end{aligned}
$$

2.1.3 Type 2: Context-Free Grammars

On type 2 grammars, or context-free grammars, the left-hand side consists of a single non-terminal symbol, and the right-hand side can be any string of terminal and/or non-terminal symbols. These grammars allows context-free grammars to generate languages that can be recognized by a pushdown automaton, which is a type of automaton that uses a stack to manage additional information [1].

An example of a context-free grammar is:

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \longrightarrow aSb, S \longrightarrow ab\}$$

Derivation of the word "aabb":

$$
\begin{aligned}
S &\longrightarrow aSb \\
aSb &\longrightarrow aaSbb \\
aaSbb &\longrightarrow aabb
\end{aligned}
$$

2.1.4 Type 3: Regular Grammars

On type 3 grammars, also known as regular grammars, represent the most constrained category within the Chomsky hierarchy. The production rules for these grammars are quite specific, each rule's left-hand side is a single non-terminal symbol, while the right-hand side is either a single terminal symbol or a terminal symbol followed by a single non-terminal symbol. This strict format allows regular grammars to generate languages that can be recognized by finite automata. [1].

An example of a regular grammar is:

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \longrightarrow aS, S \longrightarrow b\}$$

Derivation of the word "aaab":

$$S \rightarrow aS$$
$$aS \rightarrow aaS$$
$$aaS \rightarrow aaaS$$
$$aaaS \rightarrow aaab$$

This dissertation will focus on a extension of type 2 grammars: Attribute Grammars.

2.2 Attribute Grammars

Attribute grammars are an extension of context-free grammars that associate attributes with the grammar symbols and define rules for computing these attributes. These attributes provide a way to carry semantic information through the parse tree, enabling context-sensitive analysis [4].

There are two types of attributes:

- **Synthesized Attributes**: These are computed from the attributes of the children nodes in the parse tree. They propagate information up the tree.
- **Inherited Attributes**: These are computed from the attributes of the parent and/or sibling nodes. They propagate information down and across the tree.

By extending context-free grammars with attributes, attribute grammars enable more powerful and flexible language descriptions. In fact it is easy to integrate contextual aspects into a grammar, often more intuitively than through a monotonic grammar. For example, instead of needing 10 rules, we can use an attribute to achieve the same result.

2.2.1 Synthesized Attributes

Synthesized attributes are computed from the attributes of the children of a node in the derivation tree. These attributes propagate information up the tree, allowing for the evaluation of properties that depend on the structure of the subtree rooted at a particular node [4].

For example, consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$
$$P = \{S \rightarrow A, A \rightarrow aA|b\}$$

We can define a synthesized attribute length that calculates the length of the generated string. The rules for computing this attribute are as follows:

$$\text{length}(S) = \text{length}(A)$$

$$\text{length}(A \rightarrow aA) = 1 + \text{length}(A)$$

$$\text{length}(A \rightarrow b) = 1$$

Derivation of the word "aaab":

$$S \Rightarrow A, \quad \text{length}(S) = \text{length}(A)$$

$$A \Rightarrow aA, \quad \text{length}(A) = 1 + \text{length}(A)$$

$$aA \Rightarrow aaA, \quad \text{length}(A) = 1 + (1 + \text{length}(A))$$

$$aaA \Rightarrow aaaA, \quad \text{length}(A) = 1 + (1 + (1 + \text{length}(A)))$$

$$aaaA \Rightarrow aaab, \quad \text{length}(A) = 1$$

Back-substituting:

$$\text{length}(A) = 1$$

$$\text{length}(A) = 1 + 1 = 2$$

$$\text{length}(A) = 1 + 2 = 3$$

$$\text{length}(A) = 1 + 3 = 4$$

Thus, the length of the generated string **aaab** is:

$$\text{length}(S) = 4$$

In this example, the attribute length is synthesized because it is computed based on the attributes of the children nodes. For the production $A \rightarrow aA$, the length is calculated by adding 1 to the length of A. For the production $A \rightarrow b$, the length is simply 1, as b is a terminal symbol.

2.2.2 Inherited Attributes

Inherited attributes are passed from parent nodes to child nodes in the derivation tree. These attributes propagate information down and across the tree, allowing for context-sensitive analysis [4].

For example, consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow A, A \rightarrow aA|b\}$$

We can define an inherited attribute prefix that accumulates the prefix of the generated string. The rules for computing this attribute are as follows:

$$\text{prefix}(S) = ""$$

$$\text{prefix}(A \rightarrow aA) = \text{prefix}(A) + "a"$$

$$\text{prefix}(A \to b) = \text{prefix}(A) + "b"$$

Derivation of the word "aaab":

$$S \Rightarrow A, \quad \text{prefix}(S) = ""$$

$$A \Rightarrow aA, \quad \text{prefix}(A) = \text{prefix}(A) + "a"$$

$$aA \Rightarrow aaA, \quad \text{prefix}(A) = \text{prefix}(A) + "a"$$

$$aaA \Rightarrow aaaA, \quad \text{prefix}(A) = \text{prefix}(A) + "a"$$

$$aaaA \Rightarrow aaab, \quad \text{prefix}(A) = \text{prefix}(A) + "b"$$

Back-substituting:

$$\text{prefix}(S) = ""$$

$$\text{prefix}(A) = \text{prefix}(S) + "a" = "a"$$

$$\text{prefix}(A) = "a" + "a" = "aa"$$

$$\text{prefix}(A) = "aa" + "a" = "aaa"$$

$$\text{prefix}(A) = "aaa" + "b" = "aaab"$$

Thus, the accumulated prefix of the generated string is:

$$\text{prefix}(S) = "aaab"$$

In this example, the attribute prefix is inherited because it is computed based on the attributes of the parent node. For the production $A \to aA$, the prefix is calculated by appending "a" to the prefix of A. For the production $A \to b$, the prefix is calculated by appending "b" to the prefix of A.

2.2.3 Circularity

For an attribute grammar to be well-formed, the attributes associated with nonterminals at any node in the parse tree must be evaluable based on the grammar's semantic rules [3]. Since non-trivial context-free grammars can generate an infinite number of parse trees, determining well-formedness is a complex task. The primary challenge lies in circular dependencies within the attribute dependency graph. For instance, if attribute a1 depends on a2, which in turn depends on a1, neither attribute can be evaluated, leading to an evaluation deadlock.

Consider the following grammar in figure 2.2

The best way to visualize the circularity is to actually draw the dependency graph for the grammar. Figure 3 shows such a graph for the attribute grammar in figure 2.2.

The dependency graph is constructed by combining the individual dependencies from each rule in the grammar, which explains the presence of multiple List↑Value attributes. In this case, the graph contains no circular dependencies, meaning it is well-formed—i.e., all possible parse trees generated by the grammar can be evaluated using the semantic

0: Number → Sign List	(i) List↓Scale := 0
	(ii) Number↑Value := IF Sign↑Neg THEN - List↑Value ELSE List↑Value
1: Sign → +	(i) Sign↑Neg := False
2: Sign → -	(i) Sign↑Neg := True
3: List → BinaryDigit	(i) BinaryDigit↓Scale := List↓Scale
	(ii) List↑Value := BinaryDigit↑Value
4: $List_0$ → $List_1$ BinaryDigit	(i) $List_1$↓Scale := $List_0$↓Scale + 1
	(ii) BinaryDigit↓Scale := $List_0$↓Scale
	(iii) $List_0$↑Value := $List_1$↑Value + BinaryDigit↑Value
5: BinaryDigit → 0	(i) BinaryDigit↑Value := 0
6: BinaryDigit → 1	(i) BinaryDigit↑Value := $2^{\text{BinaryDigit}\downarrow\text{Scale}}$

Figure 2.2: Attribute grammar for signed binary numbers [3]

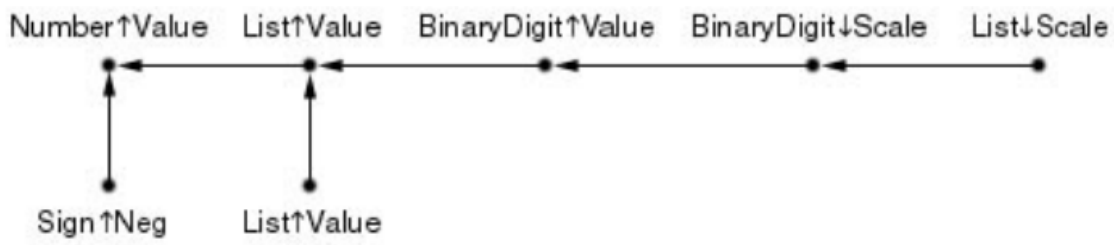


Figure 2.3: Attribute dependency graph for the grammar in figure 2 [3]

rules. A circular dependency could be introduced by reversing the arrow from Sign↑Neg to Number↑Value and adding an arrow from Sign↑Neg to the lower List↑Value as we can see in the figure bellow. In such a scenario, none of the four attributes forming the cycle could be evaluated. The core challenge in attribute grammars is detecting these cycles within the attribute dependency graph[3].

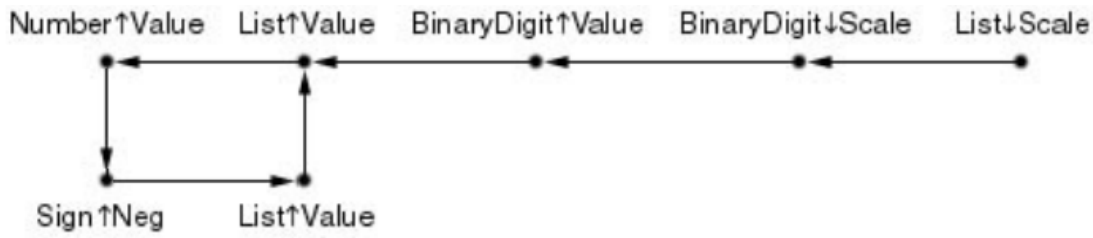


Figure 2.4: Modified attribute dependency graph with circularity

2.3 Constrained Attribute Grammars

Constrained Attribute Grammars (CAG) further enhance attribute grammars by incorporating constraints that capture spatial relationships between symbols [9]. This is essential

for recognizing multi-dimensional objects, such as mathematical formulas or organizational charts, which cannot be easily represented as one-dimensional strings.

Each production rule in a CAG can be divided in three components [9], syntatic part, semantic part, and constraints. The syntatic part defines the structure of the rule, e.g.,E→E1E2...En. On other side the semantic part specifies how to compute the attributes of the left side symbol based on the attributes of the right side symbols. Finnaly the constraints establish relationships between input symbols and rule symbols, often expressed in terms of attributes.

The parsing algorithm for CAGs leverages the concepts of keywords and relevance measures to efficiently parse multi-dimensional objects. The algorithm proceeds by selecting the most relevant keyword, applying constraints to partition the input symbols, and recursively parsing the subsets of input symbols corresponding to the non-terminal symbols in the rule [9].

2.3.1 Example of a Constrained Attribute Grammar

Consider a simple CAG for recognition mathematical expressions involving addition and division in figure 2.5

Syntactic Part	Semantic Part	Constraints
1. $S \rightarrow E \mid t$	$A(S) = A(E)$ or $A(t)$	ϕ
2. $E \rightarrow +E_1E_2$	$A(E) = A(+) \cup A(E_1) \cup A(E_2)$	ϕ , $A(x)$ is to the right of $A(+)$, and $A(x)$ is is to the left of $A(+)$
3. $E \rightarrow \div E_1E_2$	$A(E) = A(\div) \cup A(E_1) \cup A(E_2)$	ϕ , $A(x)$ is above $A(+)$, and $A(x)$ is below $A(+)$
4. $E_i \rightarrow E \mid t$	$A(E_i) = A(E)$ or $A(t)$	ϕ, for $i = 1,2$

Figure 2.5: Production rules fi r a CAG for simple mathematical expressions [9]

If we do the parsing of the mathematical expression: 3 + 5 / 2

$$
\begin{aligned}
S \Rightarrow E, \quad & A(S) = A(E) \\
E \Rightarrow E_1 + E_2, \quad & A(E) = A(+) \cup A(E_1) \cup A(E_2) \\
E_1 \Rightarrow 3, \quad & A(E_1) = A(3) \\
E_2 \Rightarrow E_3/E_4, \quad & A(E_2) = A(/) \cup A(E_3) \cup A(E_4) \\
E_3 \Rightarrow 5, \quad & A(E_3) = A(5) \\
E_4 \Rightarrow 2, \quad & A(E_4) = A(2)
\end{aligned}
$$

Now, let's back-substitute the attributes:

$$
\begin{aligned}
& A(E_4) = A(2) \\
& A(E_3) = A(5) \\
& A(E_2) = A(/) \cup A(5) \cup A(2) \\
& A(E_1) = A(3) \\
& A(E) = A(+) \cup A(3) \cup (A(/) \cup A(5) \cup A(2))
\end{aligned}
$$

Thus, the semantic attributes of the parsed string $3 + 5/2$ are:

$$A(S) = A(E) = A(+) \cup A(3) \cup A(/) \cup A(5) \cup A(2)$$

2.4 Automata

Automata theory is a fundamental area of theoretical computer science that deals with the study of abstract machines and the problems they can solve [10]. As described in the Chomsky hierarchy, different types of automata correspond to different classes of formal languages.

2.4.1 Types of Automata

2.4.1.1 Finite Automata

Finite automata (FA) are models in computer science used to recognize regular languages, they consist of a finite number of states, transitions between these states, and an input alphabet, when an input string is processed, the automaton transitions through states according to the input symbols. If it ends in an accepting state, the string is recognized as part of the language.

Due to their simplicity and efficiency, this models can be used in various applications such as in text processing, lexical analysis and in network protocols to manage and control the flow of data.

2.4.1.2 Pushdown Automata

Pushdown automata (PDA) are more powerful than finite automata and can recognize context-free languages. They extend finite automata by adding a stack, which provides additional memory, this stack allows PDAs to handle nested structures, such as parentheses in arithmetic expressions or nested loops in programming languages and used in parsing and syntax analysis in compilers [5].

2.4.1.3 Turing Machines

Turing machines (TM) are the most powerful type of automata and can recognize recursively enumerable languages, they consist of an infinite tape, a tape head that can read and write symbols, and a set of states. Turing machines can simulate any algorithm and are used to define the limits of what can be computed and are fundamental in the study of computability and complexity theory [5].

2.4.1.4 Linear Bounded Automata

Linear bounded automata (LBA) are a type of Turing machine with a tape of limited length proportional to the input size, they recognize context-sensitive languages. LBAs are more powerful than PDAs but less powerful than general Turing machines, they also are used in certain types of language recognition and computational complexity studies [5].

3

Grammar Tools

The tools for FLAT aimed at automata can be classified into two major categories: text-based automaton simulators and graphical interface-based automaton simulators. The first involves specifying a model through structured text, which must be processed using a parser, the second uses a graphical interface that allows creating, editing, and using an automaton to recognize words

The different existing tools can also support only one type of automaton or multiple types of automata, such as finite automata, pushdown automata, and Turing machines. There are also tools that deal with regular expressions and grammars, and in such cases, it is common for conversion operations to be available between automata and vice versa.

Next, some pedagogical tools are presented and classified according to the indicators described above, i.e., whether they are text-based tools or graphical interface-based tools; whether they implement only one type of automaton or multiple types, and whether they support regular expressions and grammars. We will discuss three such tools: **JFLAT**, **ANTLR**, and **CGM**.

3.1 JFLAT

JFLAT (Java Formal Languages and Automata Toolkit) is a versatile tool designed for experimenting with formal languages, it supports various computational models, including finite automata, pushdown automata, Turing machines, and different types of grammars this tool also provides a comprehensive environment for users to explore and understand the theoretical foundations of computer science. JFLAT offers several key features that enhance its utility as an educational tool as it allows users to create and visualize the computational models helping students better understand the structure and behaviour of the same models. This tool supports the conversion between different forms of automata. Additionally, JFLAT includes parsing capabilities for various types of grammars, enabling users to experiment with context-free grammars and other formal language constructs.

JFLAT also provides educational benefits by offering a visual and interactive approach to learning complex concepts, as it have the ability to visualize automata and grammars

helping users to gain a deeper understanding of theoretical concepts through hands-on experimentation, develop problem-solving skills by working through practical exercises and projects. Examples of exercises and projects include creating and testing finite automata to recognize specific patterns or languages, designing and parsing context-free grammars to explore the syntax and structure of programming languages.

3.2 ANTLR

ANTLR (ANother Tool for Language Recognition) is a parser generator used for reading, processing, and translating structured text or binary files, it offers several notable features, including the ability to generate parsers, lexers, and tree parsers from grammars, facilitating the creation of language processors. It also supports multiple programming languages, making it versatile for various development environments and also can be used to build languages, interpreters, compilers, and other language-related frameworks.

This tool help users understand the principles of syntax analysis and parsing, building parsers and compilers, and explore the language design and implementation. ANTLR can be used in various educational projects and assignments, such as creating parsers for custom programming languages, building simple compilers to learn about the different stages of compilation, and developing domain-specific languages to appreciate the practical applications of the tool in real-world scenarios.

3.3 CGM: Context-Free Grammar Manipulator

CGM (Context-Free Grammar Manipulator) is a tool developed at the University of Porto designed to assist in the manipulation and study of context-free grammars. It provides a graphical environment for editing grammars, converting them to Chomsky normal form, parsing words, and constructing parse trees interactively, it offers several key functionalities, including grammar editing, Chomsky normal form conversion, and the ability to parse words and visualize the corresponding parse trees, aiding in the understanding of grammar structures.

CGM helps users to visualize the structure and operation of context-free grammars, experiment with different grammar configurations to see how changes affect parsing and parse tree construction, and develop a deeper understanding of the concepts of grammar rules, parsing, and syntax trees.

4

OCaml-Flat and OFLAT Systems

The OCamlFLAT library and the OFLAT platform are the main componentsof this thesis, both developed in OCaml. They are already in an advanced stage of development, thanks to the work of several students and professorswho have contributed to this project.

The OCamlFLAT library consists of various modules that make up the logical part of the system, where several FLAT theory models have already been implemented. The OFLAT platform, on the other hand, is a web application that provides the system's graphical interface.

4.1 OCamlFLAT

The OCamlFLAT library is organized into modules to ensure a clearer and more structured organization. There are modules that implement models of FLAT theory, such as regular expressions, finite automata, and context-free grammars, as well as auxiliary modules for tests, error handling, and parsers. It is important to note that the modules follow a hierarchical relationship among themselves, which facilitates the maintenance and expansion of the library [7].

Some of the most important modules are:

Entity: This module represents the most abstract level of the library. The entities are the exercises and the FLAT models, each containing a type, a description, and a unique ID.

Exercise: This module is responsible for supporting exercises, which are validated using UnitTests. It extends the Entity module, inheriting its properties and methods.

Model: An abstract module that also extends Entity and introduces common functionalities to the FLAT models available in the library. Examples of methods included in this module are validate, which checks the validity of a model, and accept, which processes inputs according to the model.

Examples of modules that represent FLAT models and extend Model include:

RegularExpression: This module contains all the functionalities needed to work with regular expressions, allowing the definition and manipulation of these expressions.

FiniteAutomaton: This module implements the functionalities of a finite automaton, including the definition of states and transitions.

ContextFreeGrammar: This module offers the functionalities of a context-free grammar, allowing the definition of production rules and the analysis of strings.

PushdownAutomaton: This module contains the functionalities of a pushdown automaton, which is an extension of finite automata with an additional stack for temporary storage of information.

These modules are designed to work together, forming a robust and flexible library for the implementation and analysis of FLAT theory models.

4.2 OFLAT Platform

The OFLAT platform is an online graphical tool designed for the modeling and analysis of formal language concepts. It provides an intuitive and interactive environment where users can create and manipulate various computational models. The interface consists of a static menu on the left sidebar and a dynamic workspace on the right, which updates based on the selected options.

OFLAT supports the creation and exploration of multiple formal models, including finite automata, pushdown automata, and Turing machines. Users can visually construct these models, define states and transitions, and analyze their properties. The platform offers built-in tools for checking determinism, minimizing models, and identifying structural properties, making it a valuable resource for both learning and research.

By offering a user-friendly graphical interface, OFLAT helps users experiment with formal languages and computation theory concepts in an accessible way. It is designed to be an educational tool that simplifies complex theoretical concepts through visual representation and interactive manipulation.

5

Technology

5.1 Programming with OCaml

OCaml is a high-performance, portable functional programming language developed in 1996 by Xavier Leroy, Jérôme Vouillon, Damien Doligez, and Didier Rémy at INRIA in France [8]. Like other functional programming languages, OCaml is built around the concept of expressions, meaning computations are performed by evaluating expressions, which in turn produce values.

The example bellow, shows a few examples of OCaml computations along with their corresponding results:

```
# 4 * 2;;
- : int = 8

#let val = 4 + 3 in val + 3;;
- : int = 10
```

The first example, shows the evaluation of a expression, that produces the value 6. The second one the let ...in ... call is being used to locally define val within the expression val + 3.

5.1.1 Declarative Programming

In functional programming languages like OCaml, the code is often written in a declarative style. Declarative programming focus in describing the logic of a computation rather than controlling its flow, this approach offers several advantages, for example by abstracting away the flow of control, the code becomes simpler compared to imperative programming. Additionally, since the code aligns more closely with the underlying concepts of the project, it tends to be more understandable and less prone to logical errors. Although this may come at the cost of efficiency, it benefits the OFLAT platform and OCamlFlat library, where the theoretical concepts involved are complex.

5.1.2 Pattern Matching

One of the aspects of working with OCaml is its pattern-matching capability, since a pattern serves as an expression that defines a set of values, we can use them to match a given value against a pattern to determine whether it belongs to the set of values represented by that pattern. In OCaml, there are various ways to pattern match, as we can see in the example bellow.

```
# List.split;;
- : ('a * 'b) list -> 'a list * 'b list

# let (x, y) = List.split [(1, 2); (3, 4); (5, 6); (7, 8)];;
val x : int list = [1; 3; 5; 7]
val y : int list = [2; 4; 6; 8]
```

In this example the function takes a list of pairs and splits it into two separate lists, the x list will have the 'a element in each tuple, and the y list the 'b elements.

5.2 OCaml/JavaScript Interoperability

Interoperability between OCaml and JavaScript is made possible through the use of the `js_of_ocaml` toll [2] , which translates OCaml code into JavaScript, allowing OCaml code to run in a browser.

This tool consists of three main components:

- `js_of_ocaml` ppx: A preprocessor that extends the OCaml library, adding functionalities that facilitate integration with JavaScript.
- `js_of_ocaml` library: A library that contains bindings for various categories of JavaScript objects, enabling OCaml code to efficiently interact with JavaScript APIs and libraries.
- `js_of_ocaml` compiler: A compiler that translates OCaml code into JavaScript, allowing OCaml programs to be executed in web-based environments.

5.2.1 JavaScript Types

Types in OCaml and JavaScript are not represented in the same way in both languages. For example, a string in OCaml is a mutable array of bytes, whereas in JavaScript, strings are constant arrays of UTF-16 code points. Therefore, it is necessary to find a way to represent JavaScript types in OCaml appropriately. The table below illustrates the correspondences between the types in the two languages.

To ensure effective interoperability, it is crucial to understand how each type is handled in OCaml and JavaScript. For instance, besides strings, other types such as numbers, arrays,

and objects also have distinct representations. In OCaml, integers and floating-point numbers are treated differently compared to JavaScript, where all numbers are essentially floating-point. Arrays in OCaml are mutable, while in JavaScript, arrays can be either mutable or immutable, depending on the context.

5.2.2 Bindings

The existence and creation of bindings in `js_of_ocaml` facilitate the seamless integration of JavaScript code and libraries within the OCaml programming environment, ensuring type safety throughout the process. This powerful feature allows developers to leverage the extensive ecosystem of JavaScript while maintaining the robust type-checking capabilities of OCaml.

Although `js_of_ocaml` already includes a plethora of useful bindings, developers are not limited to these pre-existing options. It remains entirely feasible to create new bindings tailored to specific project needs, thereby enabling interaction with a wide array of additional JavaScript libraries and objects. This flexibility is particularly advantageous for projects that require the integration of specialized JavaScript functionalities or the utilization of cutting-edge libraries that may not yet be covered by the existing bindings.

By creating custom bindings, developers can ensure that their OCaml applications can interact with the latest JavaScript technologies, thereby enhancing the functionality and versatility of their projects. This approach broadens the scope of what can be achieved with `js_of_ocaml` and empowers developers to innovate and adapt to the ever-evolving landscape of web development

5.2.3 `js_of_ocaml` Modules

The `js_of_ocaml` library already includes several modules with predefined bindings, but only a few are necessary for the development of the OFLAT platform.

Below are some examples of these modules and submodules:

- Js Module: This module contains definitions of JavaScript types, functions for standard JavaScript objects, and conversion functions between OCaml and JavaScript types. It is essential for ensuring interoperability between the two languages.
- Js.Unsafe Submodule: This submodule allows for operations on JavaScript that are not safe, offering greater flexibility but requiring extra caution to avoid errors and vulnerabilities.
- `Dom_html` Module: Defines and manipulates HTML DOM elements, enabling the creation and modification of HTML structures directly from OCaml code. This module is crucial for building the graphical interface of the platform.
- Firebug Module: Allows for the visualization and debugging of objects in the browser's developer console, facilitating the development and debugging process.

5.3 Representation in OCaml

When working with OCaml and JavaScript interoperability, understanding how to represent various attributes is crucial. OCaml uses synthesized and inherited attributes to manage data and behavior in a structured way. These attributes play a significant role in the language's type system and its ability to enforce strong type safety.

5.3.1 Synthesized Attributes

These are attributes computed from the values of other attributes. For example, in a syntax tree, the type of an expression might be synthesized from the types of its sub-expressions. Synthesized attributes help in propagating information upwards in a hierarchical structure, ensuring that each node has the necessary context to perform its operations.

5.3.2 Inherited Attributes

These are attributes passed down from parent nodes to child nodes in a structure. For instance, the scope information in a syntax tree can be inherited from parent nodes to ensure proper variable resolution. Inherited attributes facilitate the flow of information downwards, providing child nodes with the context they need to function correctly.

5.3.3 Rules

The rules define how attributes are calculated. They consist of a head, which specifies the attribute being computed, and a body, which describes the computation. These rules are essential for maintaining consistency and correctness in the representation of data.

Here's a simple example to illustrate these concepts:

```
(* Example of synthesized and inherited attributes in OCaml *)
type expr =
  | Const of int
  | Add of expr * expr

(* Function to evaluate an expression with synthesized attributes *)
let rec eval = function
  | Const n -> n
  | Add (e1, e2) -> eval e1 + eval e2

(* Usage *)
let () = Printf.printf "Result of 3 + 4 is %d\n"
(eval (Add (Const 3, Const 4)))
```

In this example, the `eval` function synthesizes the result of an addition expression from its sub-expressions. This approach ensures that each part of the expression is evaluated

in a consistent and predictable manner, leveraging the power of OCaml's type system to enforce correctness.

5.4 Cytoscape

Cytoscape is an open-source software platform that, although initially designed for biological research, is now a general tool for graph visualization and analysis [11]. In this project, the OFLAT platform is used to visualize the various models supported by the application. To use this library, it is necessary to create bindings for OCaml/JavaScript interoperability.

6

Planning

6.1 Introduction

In this chapter, the work plan for this dissertation is presented, considering a period of 24 weeks (6 months).

The objective of this work is to add support for attribute grammars to the system OCamlFLAT/OFLAT. The implementation work aims to extend existing software, which already includes some relevant elements, such as support for context-free grammars and basic visualization and graphical interaction mechanisms.

6.2 Support for Attribute Grammars in the OCamlFLAT Library (8 weeks)

At the library level, several functionalities are planned:

- Representation of attributes and their calculation rules within grammars.
- Circularity testing of synthesized and inherited attributes in a grammar.
- Calculation of synthesized and inherited attributes on an already constructed derivation tree.
- Calculation of synthesized and inherited attributes during the derivation tree construction of a word, i.e., during parsing.
- Introduction of "conditions" in grammars to use attribute values to restrict the generated language.
- Integration of attribute grammars into the existing support for composite models in the library. This involves attempting various operations on attribute grammars: union, intersection, difference, concatenation, and Kleene closure.
- Integration of grammars into the existing support for pedagogical exercises in the library.

6.3 Support for Grammars in the OFLAT Web Application (6 weeks)

Attribute grammars must be integrated into the existing graphical interface. The additions should have pedagogical features, and some interesting ideas may emerge for the graphical interface.

There is already support for context-free grammars in the graphical application. That part of the code will be duplicated, refactored, and adapted. Understanding and modifying the graphical application code is not a trivial task.

What should be supported in the graphical interface for attribute grammars:

- Visualization.
- Editing.
- Visualization of the results of operations available in the library.
- Visualization of derivation trees annotated with attribute calculations. In a more ambitious version, attribute calculation could be animated.
- Integration into the existing support for composite models in the graphical application.

6.4 Pedagogical Exercises (1 week)

Create a collection of interesting pedagogical examples and encode them into a set of exercises in the format supported by the OCamlFLAT/OFLAT system.

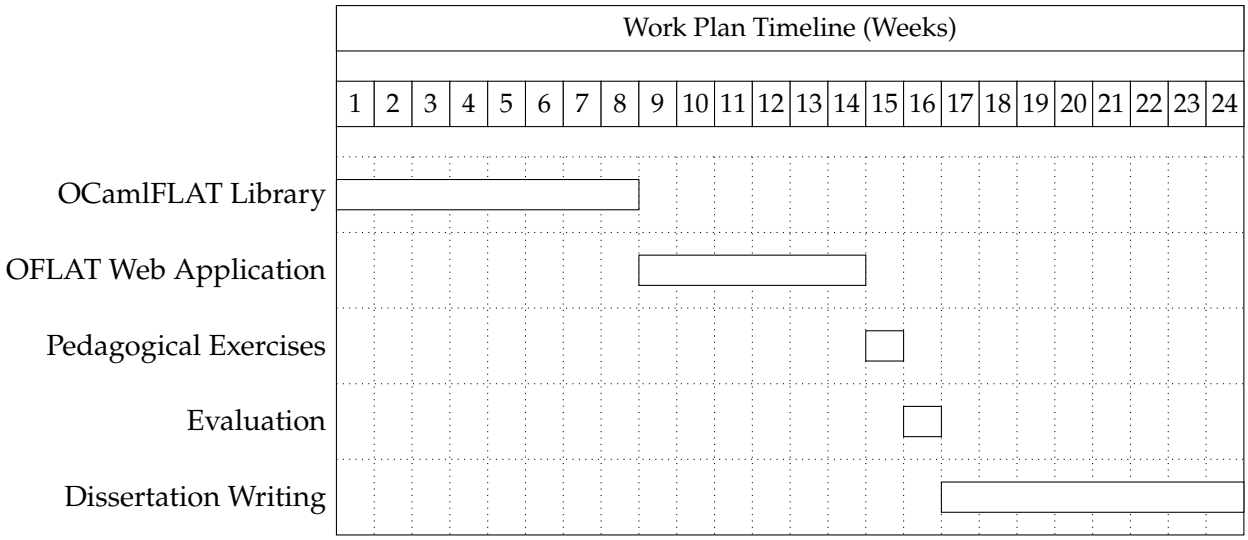
6.5 Evaluation (1 week)

- The correctness of the newly implemented code will be assessed through a set of unit tests, which will be written alongside code development.
- Analyzing the system's limitations by creating examples that pose challenges due to size or execution time.
- Seeking feedback from potential users of the system, particularly regarding the graphical interface. For example, teachers and students of the TC course.

6.6 Dissertation Writing (8 weeks)

Ideally conducted in small increments, concurrently with other tasks. Two months are considered for the production of the final document. It is important to have a near-final version ready two weeks before the submission deadline to facilitate inviting an examiner.

6.7 Gantt Chart



Bibliography

[1] N. Chomsky. "Three models for the description of language". In: *IRE Transactions on Electronic Computers* EC-8.1 (1956) (cit. on pp. 3–5).

[2] B. Demange et al. *From Bytecode to JavaScript: The Js of ocaml Compiler*. 2012. URL: https://citeseerx.ist.psu.edu/document?doi=1b87d68c7c4f7b897eeb09804657225f6f8f762f&repid=rep1&type=pdf (cit. on p. 18).

[3] C. Gordon. *The Circularity Problem for Attribute Grammars*. Accessed: 2025-01-17. 2000. URL: https://courses.cs.washington.edu/courses/cse531/00au/gordon.pdf (cit. on pp. 8, 9).

[4] D. E. Knuth. "Semantics of Context-Free Languages". In: *Theory of Computing Systems* 2.2 (1968), pp. 127–145 (cit. on pp. 6, 7).

[5] D. E. Knuth. "Semantics of Context-Free Languages". In: *Theory of Computing Systems* 2.2 (1968), pp. 127–145 (cit. on p. 12).

[6] J. M. Lourenço. *The NOVAthesis LaTeX Template User's Manual*. NOVA University Lisbon. 2021. URL: https://github.com/joaomlourenco/novathesis/raw/main/template.pdf (cit. on p. i).

[7] A. Mig and jmggoncalves. *OFLAT: A Formal Language Toolkit*. https://github.com/git-amd/OFLAT. Accessed: 2025-02-02. 2025 (cit. on p. 15).

[8] *OCaml History*. Accessed: 2025-01-17. URL: https://ocaml.org/about (cit. on p. 17).

[9] G. Pagallo. "Constrained Attribute Grammars for Recognition of Multi-dimensional Objects". In: (). Accessed: 2025-01-17. URL: https://link.springer.com/content/pdf/10.1007/BFb0033254.pdf (cit. on pp. 9, 10).

[10] A. Ravara. *Slides da disciplina de teoria da computação*. Rel. téc. FCT-UNL, 2020 (cit. on p. 11).

[11] G. Shannon et al. *Cytoscape: A Software Environment for Integrated Models of Biomolecular Interaction Networks*. Springer, 2003 (cit. on p. 21).